

MS1 – Similar Practice Questions

These questions retain the same patterns and assessment style as the supplied papers while changing the domain, names, values, and wording. The source Python paper uses independent Pandas, NumPy, and OOP tasks; the SQL paper uses window-function reporting tasks. ■filecite■turn0file0■L4-L11■ ■filecite■turn0file1■L11-L23■

PYTHON QUESTION 1

Python Question: Hospital Equipment Service Analyzer

Marks: 20 **Difficulty:** Medium **Technology:** Pandas

1. Problem Statement

A hospital network tracks service inspections for critical medical equipment. Build a Pandas analyzer that creates the service table, cleans invalid records, flags excessive service downtime, summarizes equipment performance, identifies low-usage equipment, and calculates completed-service cost by department.

Complete the class in solution.py. Do not change the class name, method names, parameter lists, or return types. Every analysis method receives a complete DataFrame and must work independently.

2. Input Schema

Column	Meaning
ServiceID	Unique service record ID
EquipmentID	Medical equipment identifier
Department	Hospital department
ServiceDate	Date supplied as input
UsageHours	Equipment usage hours
DowntimeHours	Downtime during service period
ServiceStatus	Completed or Scheduled

Keep ServiceDate as supplied; datetime conversion is not required.

3. Class

```
class HospitalEquipmentAnalyzer:
    # No constructor state is required
```

4. Operations

Operation 1: Create the service DataFrame

```
def create_services_df(self, service_data: list) -> pd.DataFrame:
```

- Create one row for each inner list.
- Use the seven required columns in the exact schema order.
- Preserve input row order and return a Pandas DataFrame.

Operation 2: Clean service data

```
def clean_service_data(self, df: pd.DataFrame) -> pd.DataFrame:
```

- Retain only rows with no missing values.
- UsageHours must be strictly greater than 0.
- DowntimeHours must be greater than or equal to 0.
- ServiceStatus must be either 'Completed' or 'Scheduled'.
- Return a copy with the original seven-column order, retained row order, and a zero-based index.
- Do not modify the input DataFrame.

Operation 3: Add the attention flag

```
def add_attention_flag(self, df: pd.DataFrame, downtime_threshold: float) -> pd.DataFrame:
```

- Return a copy of df.
- Add NeedsAttention as the last column.
- Store integer 1 when DowntimeHours > downtime_threshold; otherwise store integer 0.
- Equality with the threshold must produce 0.

Operation 4: Equipment performance summary

```
def equipment_performance_summary(self, df: pd.DataFrame) -> pd.DataFrame:
```

- Group by EquipmentID.
- Return EquipmentID, ServiceCount, TotalUsageHours, and AverageDowntime.
- AverageDowntime is the mean of DowntimeHours rounded to two decimals.
- Sort by EquipmentID ascending and return a zero-based index.

Operation 5: Find low-usage equipment

```
def low_usage_equipment(self, df: pd.DataFrame, usage_threshold: float) -> pd.DataFrame:
```

- Sum UsageHours by EquipmentID.
- Keep equipment whose total is strictly less than usage_threshold.
- Equality with the threshold must not be included.
- Return EquipmentID and TotalUsageHours, sorted by EquipmentID with a zero-based index.

Operation 6: Department service-cost summary

```
def departmental_service_cost(self, df: pd.DataFrame) -> pd.DataFrame:
```

- Use only rows where ServiceStatus == 'Completed'.
- Calculate row cost as DowntimeHours * 300.0.
- Sum the cost by Department.
- Return Department and ServiceCost, sorted by Department with a zero-based index.

Example: a completed service with 2.0 downtime hours contributes 600.0.

PYTHON QUESTION 2

Python Question: Aquarium Water Quality Analyzer

Marks: 20 **Difficulty:** Medium **Technology:** NumPy

1. Problem Statement

An aquarium research centre records water temperature readings in degrees Celsius. Implement five independent NumPy functions that create, validate, summarize, categorize, and scan those readings.

A valid recorded temperature is from 12.0 through 30.0, inclusive.

2. Operations

Operation 1: Create a temperature array

```
def create_water_temperature_array(values: list) -> np.ndarray:
```

- Convert values to a one-dimensional NumPy array.
- The returned dtype must be np.float64.
- Preserve input order.

Operation 2: Validate a temperature array

```
def validate_water_temperature_array(arr: np.ndarray) -> bool:
```

- Return Python boolean True only when arr is non-empty.
- Its dtype is a numeric NumPy subtype.
- Every value is from 12.0 to 30.0, inclusive.
- Return False for an empty, non-numeric, or out-of-range array.
- A non-numeric array must not cause an exception.

Operation 3: Compute temperature statistics

```
def compute_water_temperature_stats(arr: np.ndarray) -> tuple:
```

- Return (mean, standard_deviation, maximum, minimum).
- Use np.mean and np.std with default ddof=0.
- Round all four values to two decimal places and return Python float values.

Operation 4: Categorize temperatures

```
def categorize_water_temperatures(arr: np.ndarray) -> np.ndarray:
```

Category	Temperature Range
Cold	12.0 <= value <= 18.0
Optimal	18.0 < value <= 26.0
Warm	26.0 < value <= 30.0
Invalid	value below 12.0 or above 30.0

Return a NumPy string array with one category per input value.

Boundary example: [12, 18, 18.1, 26, 26.1, 30, 30.1] → [Cold, Cold, Optimal, Optimal, Warm, Warm, Invalid].

Operation 5: Find the longest warm streak

```
def longest_warm_streak(arr: np.ndarray) -> int:
```

- A warm reading satisfies $26.0 < \text{value} \leq 30.0$.
- Return the greatest number of consecutive warm readings.
- Any other value, including an invalid value, breaks the current streak.
- Return 0 when no warm reading exists.

PYTHON QUESTION 3

Python Question: Library Book Loan Registry

Marks: 20 **Difficulty:** Medium **Technology:** Python OOP

1. Problem Statement

A library maintains an in-memory registry of books currently loaned to members. Build a small registry that can register a loan, change its member, look up one loan, and list loans belonging to a particular member.

Complete the class in solution.py. Do not change the class name, state attribute, method names, parameter lists, or return types.

2. Class and State

```
class LibraryLoanRegistry:
    def __init__(self):
        self.loans = {}
```

self.loans maps each loan_id to a dictionary with exactly these keys:

```
"book_title": str
"member_name": str
"status": "On Loan"
```

3. Operations

Operation 1: Register a book loan

```
def register_loan(self, loan_id: str, book_title: str, member_name: str) -> dict:
```

- If loan_id already exists, raise ValueError('Loan already registered') without changing the stored record.
- Otherwise store book_title, member_name, and status 'On Loan'.
- Return the complete updated self.loans dictionary.

Operation 2: Change the loan member

```
def change_member(self, loan_id: str, new_member_name: str) -> dict:
```

- If loan_id does not exist, raise KeyError('Loan not found').
- Update only member_name.
- Preserve book_title and status.
- Return the complete updated self.loans dictionary.

Operation 3: Get loan details

```
def get_loan_details(self, loan_id: str) -> dict:
```

- If loan_id does not exist, raise KeyError('Loan not found').
- Return the stored details dictionary for that loan.

Operation 4: List loans by member

```
def loans_by_member(self, member_name: str) -> list:
```

- Return loan IDs whose stored member_name exactly equals member_name.
- Matching is case-sensitive.
- Preserve insertion order of self.loans.
- Return an empty list when there are no matches.

SQL QUESTION 1

SQL Question: Average Delivery Time by Route

Marks: 20 Difficulty: Medium Database: DELIVERY_TRACKING_DB

Schema Details

Table	Column	Type	Description
Routes	route_id	INT (PK)	Unique route ID
Routes	route_name	VARCHAR(100)	Name of route
Drivers	driver_id	INT (PK)	Unique driver ID
Drivers	driver_name	VARCHAR(100)	Driver name
Deliveries	delivery_id	INT (PK)	Unique delivery ID
Deliveries	route_id	INT (FK)	Route used
Deliveries	driver_id	INT (FK)	Driver who delivered
Deliveries	delivery_time	DECIMAL(6,2)	Delivery time in minutes
Deliveries	status	ENUM	Completed, Delayed, Cancelled

Problem Statement

Generate a report for every completed delivery showing the route name, driver name, delivery time, and the average delivery time of all completed deliveries within the same route.

- Use AVG() as a window function with PARTITION BY route_id.
- Only status = 'Completed' should be included.
- Delayed and Cancelled deliveries must not appear and must not affect the route average.
- Round the calculated average delivery time to 2 decimal places.
- Sort by route_name, delivery_time, driver_name.

Required Output Columns

- route_name
- driver_name
- delivery_time
- avg_route_time

Sample Data

delivery_id	route_id	driver_id	delivery_time	status
301	1	1	42.50	Completed
302	1	2	39.00	Completed
303	1	3	46.00	Completed
304	1	4	55.00	Delayed
305	2	1	61.00	Completed
306	2	2	58.50	Completed
307	2	3	63.50	Completed
308	2	4	70.00	Cancelled

SQL QUESTION 2

SQL Question: Days Between Patient Appointments

Marks: 20 Difficulty: Medium Database: PATIENT_APPOINTMENTS_DB

Schema Details

Table	Column	Type	Description
Patients	patient_id	INT (PK)	Unique patient ID
Patients	patient_name	VARCHAR(100)	Patient name
Appointments	appointment_id	INT (PK)	Unique appointment ID
Appointments	patient_id	INT (FK)	Patient attending
Appointments	appointment_date	DATE	Appointment date
Appointments	fee	DECIMAL(10,2)	Appointment fee

Problem Statement

For each appointment, calculate the number of days that have passed since the patient's immediately previous appointment.

- Process appointments separately for each patient using `patient_id`.
- Use `LAG()` to retrieve the immediately previous `appointment_date` for each patient.
- Determine appointment sequence using `appointment_date` and then `appointment_id`.
- Calculate the number of days between the current and previous appointment.
- For the first appointment of each patient, `days_since_previous_appointment` should be `NULL`.
- Sort by `patient_name`, `appointment_date`, `appointment_id`.

Required Output Columns

- `patient_name`
- `appointment_id`
- `appointment_date`
- `days_since_previous_appointment`

Sample Data

patient_id	patient_name
1	Meera
2	Rohan
3	Sahil

appointment_id	patient_id	appointment_date	fee
801	1	2026-08-02	700.00
806	1	2026-08-08	850.00
814	1	2026-08-21	700.00
902	2	2026-08-04	600.00
909	2	2026-08-18	750.00
1001	3	2026-08-06	900.00
1008	3	2026-08-16	900.00